

Guion de exposición

MesaTech Cloud · Evaluación Parcial N.º 1 · 10 minutos

Ari Araya · Nicolas Barra · Giannina Guerrero · Skarlett Tropan

Docente Paulo Andrés Berríos Díaz

Regla del profesor: no abrir la aplicación en vivo. Solo diapositivas y capturas. El docente puede pedir una demostración otro día.

Presentación: entregables/presentacion/index.html (18 láminas). En el navegador, tecla N muestra el mismo texto. Este PDF es la copia para ensayar en papel.

Relato: primero cómo se configuró (Entra → Gateway → EC2), después cómo se ve funcionando en React. Mira la captura, no el título. «Si preguntan» no se recita.

Ritmo: el texto de DECIR está escrito para leerse con pausa, no de corrido. Entre dos frases, mira la captura dos segundos. Si al ensayar se pasan de 10:00, corten la última frase del bloque, no la idea. Si terminan en 8 minutos, están leyendo demasiado rápido.

Tiempos y palabra

Reloj	Lámina	s	Quién	Captura
0:00–0:30	01. Portada	30	Giannina	Portada
0:30–1:00	02. Agenda	30	Nico	—
1:00–1:35	03. Problema	35	Skarlett	Canales
1:35–2:10	04. Propuesta	35	Skarlett	Flujo de estados
2:10–2:55	05. Arquitectura	45	Nico	Diagrama us-east-1
2:55–3:30	06. Registro	35	Ari	api-cloud-native + URI
3:30–4:10	07. Entra ID	40	Ari	access_as_user + roles
4:10–4:50	08. JWT	40	Nico	401 BFF sin Bearer
4:50–5:30	09. Gateway	40	Giannina	HTTP API mesatech-ev1
5:30–6:05	10. Rutas	35	Giannina	Rutas + integración :8080
6:05–6:50	11. Perímetro	45	Giannina	Authorizer + Network
6:50–7:25	12. Despliegue	35	Nico	BFF :8080 y MS :8081
7:25–8:00	13. Datos	35	Nico	Docker + folios CREADA
8:00–8:35	14. Red	35	Nico	Timeout :8081 + SG 5432
8:35–9:20	15. Aplicación	45	Skarlett	Login Entra + cliente
9:20–10:05	16. Reglas	45	Skarlett	403 cliente + admin
10:05–10:40	17. Pruebas	35	Equipo	Cinco prints ya vistos
10:40–11:10	18. Cierre	30	Nico	Preguntas

Habla presupuestada: 670 s (11:10), con la lámina 06 de registro de api-cloud-native. Ese reloj asume pausa en cada captura. Si leen de corrido, caen cerca de 9:30. Si el docente pide 10:00: unir Rutas con Gateway y dejar Pruebas en una frase. No eliminar 401 ni 403.

Texto hablado (ensayar en voz alta)

01. Portada · Giannina · 30 s · 0:00–0:30

DECIR

Buenos días. Somos MesaTech Cloud. MesaTech atiende soporte tecnológico: el cliente hoy escribe por correo, llama o manda un chat, y el operador no ve esas solicitudes en un solo lugar. Nosotros armamos un portal web. La persona entra con su cuenta, deja una solicitud y esa solicitud queda con un estado, un dueño y un historial. En estos minutos vamos a mostrar dos cosas. Cómo se configuró esa puerta: quién entra, por dónde entra y dónde corre. Y, con capturas, que funciona. No vamos a abrir la aplicación ahora.

SI PREGUNTAN

Presenta el producto, no el curso. Habla despacio. Un segundo después de «MesaTech Cloud». Si piden ver la app, otro día. El front puede ir en localhost; la API pasa por el Gateway. Giannina abre. Nico cierra.

02. Agenda · Nico · 30 s · 0:30–1:00

DECIR

El orden es el del producto, no una lista para recitar. Primero el caso: por qué el soporte está partido y qué portal proponemos. Después el mapa: React no llama a los servidores; habla solo con el Gateway. Luego cómo se configuró, en tres capas: Entra ID, la API en AWS, y EC2 con la base. Al final, React funcionando: un login, tres perfiles, y las pruebas que ya se ven en las capturas. Cada bloque tiene dos o tres prints. Skarlett empieza con el problema.

SI PREGUNTAN

No enumeres del uno al ocho. Di el camino: caso, mapa, configuración, React. Si preguntan un detalle, «lo vemos en la captura de esa capa». Nico toma acá y le pasa a Skarlett.

03. Problema · Skarlett · 35 s · 1:00–1:35

DECIR

Hoy el soporte de MesaTech no tiene un solo registro. El cliente escribe por correo, llama por teléfono o manda un chat. El mismo incidente queda en tres hilos distintos. El operador no ve una cola: no sabe cuántas solicitudes hay abiertas ni en qué etapa está cada una. Tampoco hay un dueño claro ni una fecha de último cambio. El cliente, entonces, vuelve a preguntar: en qué quedó mi solicitud. El problema no es que falte un sistema más. El problema es que no hay un estado común que todos puedan consultar.

SI PREGUNTAN

Trazabilidad: estado, dueño y fecha de un ticket sin reconstruir el hilo. MesaTech es una PYME de soporte. Skarlett habla el caso porque redactó el informe. Si te aceleras, para en «tres hilos» y mira las tres tarjetas.

04. Propuesta · Skarlett · 35 s · 1:35–2:10

DECIR

La propuesta es un portal autenticado con Entra ID. Tres perfiles: cliente, operador y administrador. El cliente crea y ve solo las suyas. El operador avanza la cola. El administrador mantiene el catálogo y también ve la bandeja. Cada solicitud nace con categoría —INCIDENTE, CONSULTA o SUGERENCIA— y con prioridad —BAJA, MEDIA o URGENTE—. Y tiene un flujo de estados: CREADA, ASIGNADA, EN_PROCESO, RESUELTA y CERRADA. También existe CANCELADA, para un caso que no se va a atender. A partir de acá no hablamos más del caso: mostramos cómo se configuró esa puerta y cómo se ve en React.

SI PREGUNTAN

No recites los cinco estados dos veces. Una pasada basta. Si preguntan por CANCELADA: anula un caso que no se atiende; no es el cierre normal. La evidencia de configuración viene en las láminas siguientes.

05. Arquitectura · Nico · 45 s · 2:10–2:55

DECIR

Este es el mapa de lo que configuramos. Lo recorro una vez, de la persona hasta la base. La persona abre React. El login no es nuestro: es Microsoft Entra ID. React obtiene un Access Token y lo manda solo al API Gateway. El Gateway es la única puerta pública. Desde ahí la petición llega al BFF, en el puerto 8080. El BFF no tiene base de datos: orquesta. Si el front pide solicitudes, llama al microservicio 8081. Si pide catálogo, llama al 8082. Postgres está en Docker, en otra EC2, y solo esos microservicios le escriben. Ahora vienen las capturas de cada capa. No vamos a dictar IPs.

SI PREGUNTAN

BFF, Backend for Frontend: orquesta lo que necesita la UI y aplica la política. No persiste tickets. Separar catálogo y solicitudes es un límite de dominio. Tres EC2: datos, microservicios y BFF. NIC-01 muestra solo la EC2 de datos; no digas que se ven las tres. Señalá las cajas, no leas la lámina entera.

06. Registro · Ari · 35 s · 2:55–3:30

DECIR

El primer paso de identidad fue registrar la API. En Entra ID se creó una aplicación nueva, no la SPA: se llama api-cloud-native. En la captura de la izquierda está el formulario: un solo inquilino, y sin URI de redirección, porque esta app no es el login del navegador. A la derecha se creó el URI de id. de aplicación: api:// más el Id. de aplicación. Ese URI es el prefijo de los scopes. El consentimiento y los roles vienen en la lámina siguiente.

SI PREGUNTAN

No leas los UUID. Redirect URI vacío a propósito: vive en la SPA. El URI se acepta el que propone Azure. Evidencia: Word de Ari, Parte A, Registro de Aplicación y Creación de URI de id de Apps. Si preguntan por el audience: es el Id. de aplicación de esta API, no el de React.

07. Entra ID · Ari · 40 s · 3:30–4:10

DECIR

La identidad se configuró en dos registros. Primero la SPA, que es el React. Después la API, que se llama api-cloud-native, con el scope access_as_user. Ese scope es el permiso que pide React para hablar a nombre de la persona. En esta captura de la izquierda está el consentimiento de administrador: el tenant autoriza ese permiso una vez, no cada usuario. A la derecha hay tres usuarios de prueba, cada uno con un rol: cliente, operador o administrador. Ese rol viaja dentro del Access Token. El BFF lo lee y decide qué puede hacer esa persona. El ID Token no se manda al Gateway: solo describe a quién inició sesión.

SI PREGUNTAN

Access Token llama APIs. ID Token describe a la persona. iss es Entra; aud es api-cloud-native. No leer IDs ni contraseñas. El Word de Ari tiene redirect y token v2; en la oral bastan estas dos capturas. Si te apuras, para en el consentimiento y después en los roles.

08. JWT · Nico · 40 s · 4:10–4:50

DECIR

Ari dejó en el BFF el issuer y el audience que deben coincidir con Entra. Esta captura es mía. Hice curl al puerto 8080 sin encabezado Authorization, sin Bearer. La respuesta es 401. Eso significa: no sé quién eres. Hay dos puertas, y las dos se piden. El JWT Authorizer corta en el Gateway, en el borde. El BFF es Resource Server: vuelve a validar firma, emisor, audiencia y expiración. No se fía solo de que el Gateway haya dejado pasar. 401 es identidad ausente o inválida. 403 es otra cosa: te conozco y no puedes. Esa diferencia la vamos a ver en React.

SI PREGUNTAN

JWT: token firmado, no una sesión. Bearer: Authorization: Bearer más el token. Token inválido o expirado también da 401. Esa prueba no está en Evidencias/Ninna; si preguntan, está en el informe. No leas el curl línea por línea: di el resultado.

09. Gateway · Giannina · 40 s · 4:50–5:30

DECIR

Paso a AWS. El primer paso fue crear una sola HTTP API, con el nombre mesatech-ev1, en la región us-east-1: la misma de las EC2. En esta captura se ve la API creada. React no guarda la IP de ningún servidor. React usa la Invoke URL del Gateway. Esa URL es execute-api.amazonaws.com. En el front, la variable REACT_APP_API_BASE_URL apunta ahí. El Gateway reenvía al BFF, puerto 8080. Nunca a la IP de solicitudes ni a la de catálogo. Si el microservicio cambia de instancia, el front no se toca: sigue hablando con la misma puerta.

SI PREGUNTAN

HTTP API no es REST API clásica. Si preguntan por qué no REST API: menos costo y JWT Authorizer nativo, suficiente para EV1. No dictes la Invoke URL completa. Un segundo mirando la captura, después la frase del BFF.

10. Rutas · Giannina · 35 s · 5:30–6:05

DECIR

Segundo paso: las rutas. Solicitudes, catálogo y usuario salen por esa misma HTTP API. No hay un Gateway para cada microservicio. En la captura de la izquierda están las rutas. En la de la derecha, la integración: HTTP hacia el BFF, puerto 8080. El path no se reescribe. Si React pide /v1/solicitudes, el BFF recibe /v1/solicitudes. También existe /api/usuario: le dice a React quién está logueado y con qué rol. Conviven v1 y v2. React usa v1, que es un arreglo. v2 envuelve version, usuario y solicitudes, para no romper el front el día que el contrato crezca.

SI PREGUNTAN

Path sin reescritura: el Gateway no traduce la URL. No hay print de v1/v2 en Evidencias; el contrato está en el informe. Señala izquierda rutas, derecha integración. No leas todos los paths.

11. Perímetro · Giannina · 45 s · 6:05–6:50

DECIR

Tercer paso: el perímetro. Las rutas de negocio llevan JWT Authorizer. Sin Bearer, el Gateway responde 401 y la petición no llega al BFF. Eso se configura en la consola; acá se ve el resultado. Ahora la derecha: ya está funcionando. El navegador está en localhost, el React de desarrollo. La API está en AWS. El Network muestra dos llamadas. OPTIONS 204: el navegador pregunta si ese origen puede mandar Authorization; el Gateway dice que sí. GET 200: la llamada de negocio respondió bien. CORS no es un login. CORS es la regla del navegador. Nosotros la configuramos; esta captura es la prueba de que quedó bien.

SI PREGUNTAN

Preflight OPTIONS: el navegador pregunta antes del GET. 204 permitido. 200 negocio ok. La consola CORS es NIN-05 en Evidencias; no hace falta abrirla. El Authorizer corta en el borde; el BFF vuelve a validar. Para dos segundos en el Network: 204 y 200 se tienen que ver.

12. Despliegue · Nico · 35 s · 6:50–7:25

DECIR

En EC2 se levantaron tres capas, no una sola. El BFF está en 8080. Ahí llega el Gateway. El BFF es Resource Server y no tiene JPA: no escribe base. A la derecha, los microservicios: solicitudes en 8081 y catálogo en 8082. Ellos sí persisten. El Gateway apunta al 8080, no al 8081. Si alguien llama al 8081 desde Internet, no debería entrar: eso lo vemos en red. No recitamos IPs. La guía permite una sola instancia; separamos para aislar datos, microservicios y BFF.

SI PREGUNTAN

JPA es acceso a base desde Java. NIC-01 no muestra las tres instancias: solo la de datos. NIC-05 sí muestra 8080, 8081 y 8082 escuchando. Señala BFF, después MS. No inventes un diagrama de las tres EC2 si no está en lámina.

13. Datos · Nico · 35 s · 7:25–8:00

DECIR

La persistencia es PostgreSQL 16, en Docker, dentro de EC2. La guía lo permite; no usamos RDS. Hay dos bases: solicitudes y catálogo, porque son dos dominios. Cambiar una prioridad del catálogo no se mezcla con cambiar el estado de un ticket. El BFF no escribe en ninguna de las dos. A la derecha ya se ve el funcionamiento. Skarlett consultó de nuevo y los folios 0001 y 0002 siguen en CREADA. CREADA es el estado inicial: el ticket nació y todavía nadie lo tomó. Si se apagara el front, esos registros seguirían en Postgres.

SI PREGUNTAN

Folio: identificador visible. Docker versus RDS: misma persistencia, más simple en Learner Lab. La captura de UI es de Skarlett; Nico habla la base. Un segundo en Docker, un segundo en los folios.

14. Red · Nico · 35 s · 8:00–8:35

DECIR

Se cerró la red. Hay que separar dos pruebas, porque las dos se piden. Primera: desde Internet, el puerto 8081 da timeout. La petición ni entra a solicitudes. Segunda: el puerto 5432 de Postgres solo acepta tráfico del security group de los microservicios. El security group es el firewall de la instancia. 401 es la aplicación diciendo no sé quién eres. Timeout es la red diciendo este puerto no es público. No es lo mismo. El BFF recibe al Gateway en 8080; los microservicios y la base no están abiertos a Internet.

SI PREGUNTAN

0.0.0.0/0 significaría abierto a Internet; MS y base no lo están. NIC-09 tiene SG de BFF, MS y DB. En lámina está el de la base. Si preguntan por el BFF: recibe al Gateway, no a cualquier cliente en 8081.

15. Aplicación · Skarlett · 45 s · 8:35–9:20

DECIR

Así se ve funcionando. Ari deja el login de la izquierda: no es un usuario que hayamos inventado en React. MSAL abre Entra y la persona entra con la cuenta del tenant. Cuando vuelve, React ya tiene nombre, rol y el botón Cerrar sesión. A la derecha está el cliente. Solo ve Mis solicitudes. Puede crear y consultar las suyas. No ve la bandeja general ni el catálogo. Un mismo login, tres experiencias. El operador vería la cola y los estados. El administrador vería eso más el catálogo. El menú se arma con el rol que viene en el token, no con un dato que escriba el front.

SI PREGUNTAN

SKA-UI02 es el formulario de alta, pero es demo local sin Entra: no la pongas. Acá se usa ska-cliente, ya autenticado. Si te aceleras, para en el login y después en «solo ve Mis solicitudes».

16. Reglas · Skarlett · 45 s · 9:20–10:05

DECIR

Autenticado no es autorizado. Este cliente de la izquierda ya tiene un JWT válido: Entra lo conoce. Aun así pide GET /v1/solicitudes, la bandeja de todos. El BFF responde 403. Te conozco y no puedes. El cliente solo puede /v1/solicitudes/mias. A la derecha, el administrador sí ve bandeja y catálogo. El operador comparte la bandeja; no mantiene el catálogo. Hay una tercera regla, de negocio: 409. No se pasa a RESUELTA si el ticket no está EN_PROCESO. No se pasa a CERRADA si no está RESUELTA. 401 es sin identidad. 403 es rol. 409 es el flujo de estados.

SI PREGUNTAN

SKA-UI04 y SKA-UI05 son demo local: no las mostremos. El 403 y el admin de esta lámina sí van con Entra. La UI también limita el selector. Si preguntan por 409 sin captura autenticada: documentado; no hay esa captura en la oral.

17. Pruebas · Equipo · 35 s · 10:05–10:40

DECIR

La guía pide nueve chequeos. En esta oral ya se vieron con print, no de memoria. Nico: 401 del BFF sin token, y timeout a 8081. Giannina: GET 200 y CORS, OPTIONS 204, en el Network. Skarlett: 403 del cliente a la bandeja, y los folios que siguen en CREADA. Ari: el token sale de Entra, con rol. Lo que no tiene captura en esta presentación está escrito en el informe. Si preguntan por un chequeo que no está en lámina, la frase es: está documentado; no trajimos esa captura autenticada a la oral.

SI PREGUNTAN

Los nueve: sin sesión; 401 Gateway; 401 JWT inválido; 200; 403; CORS; 409; v1/v2; persistencia. No hay en Evidencias: 401 Gateway, JWT inválido y cuerpo v1/v2. 409 y bandeja operador son demo local. No recites los nueve: nombra los que sí se vieron.

18. Cierre · Nico · 30 s · 10:40–11:10

DECIR

Cierro con tres cosas que quedan demostradas, no con un resumen largo. Identidad: Entra emite el JWT y, sin token, Gateway y BFF responden 401. Perímetro: React solo habla con el Gateway; los microservicios no son públicos. Dominio: 403 es rol, 409 es regla de estado, y Postgres persiste. Ari configuró Entra. Giannina, el Gateway. Yo, EC2 y la red. Skarlett, React y las reglas. Si quieren ver la aplicación corriendo, se coordina otro día. Preguntas.

SI PREGUNTAN

No abrir la app ahora. El MVP no es un ITSM completo; cumple el flujo mínimo de la EP1. Después de «Preguntas», cállense y esperen. Si nadie pregunta, no rellenen.

Qué llevar

- Informe formal: MesaTech_EV1_Informe_Formal.docx
- Esta presentación HTML a pantalla completa (F). Tecla N para las notas.
- Este PDF impreso o en el celular, por si el panel se tapa.

Evidencias: qué hay y qué no mostrar

- En la oral (con Entra o AWS real): registro de api-cloud-native, URI de id. de aplicación, consent, roles, login, 401 BFF, HTTP API, rutas, integración, Authorizer, Network 204/200, BFF/MS, Docker, persistencia, timeout, SG, 403, admin.
- Hay en carpeta, pero son demo local sin Entra: SKA-UI02 registrar, SKA-UI04 bandeja, SKA-UI05 / ska-409. No las pongan en la oral.
- Hay y no están en lámina: NIN-05 CORS (se ve el resultado en Network). NIC-01 muestra solo la EC2 de datos, no las tres.
- No está en Evidencias: 401 del Gateway, JWT inválido y el cuerpo de v1/v2.
- Antes de entregar: sacar contraseñas e IDs del Word de Ari (Screenshots EP1.docx).

Qué no decir / no mostrar

- Tokens Bearer completos, contraseñas, .pem o .env.
- «Les muestro la app ahora».
- Leer el informe en voz alta.
- IPs privadas, Tomcat, Hibernate o «señalar de izquierda a derecha».

Ensayo (30 minutos el día anterior)

1. Cronómetro de 10:00, sin pausas.
2. Cada uno solo su bloque; el siguiente se para al borde.
3. Si alguien se pasa 20 segundos, recortar ejemplos, no la idea.
4. Tres preguntas de práctica: ¿qué valida el Gateway?, ¿por qué 403 y no 401?, ¿dónde persisten las solicitudes?